

Banking Service Management System

Software Documentation

Framework	Django 5.2.17
Language	Python 3.10.0
API Layer	Django REST Framework
Database	SQLite
Frontend	HTML, CSS, JavaScript
Django App	Accounts
User Roles	Admin / Staff and Customer
Deployment	Localhost (http://127.0.0.1:8000/)

Prepared as a complete technical and functional reference for the design, implementation, and operation of the Banking Service Management System.

Table of Contents

1. Project Overview
2. Technology Stack
3. System Objectives
4. System Users
5. Main Modules
6. User Login Module
7. Customer Management
8. Account Management
9. Transaction Management
10. Transaction Database
11. Other-Bank Transactions
12. REST API
13. Get Account API
14. Get All Accounts API
15. Create Transaction API
16. Serializers
17. Database Design
18. Data Relationship
19. Validation and Error Handling
20. Error Handling Examples
21. API Testing
22. Running the Project
23. Project Structure
24. Application Workflow
25. Security
26. Advantages
27. Limitations
28. Future Enhancements
29. Testing Checklist
30. API Summary
31. Conclusion

1. Project Overview

The Banking Service Management System is a web-based banking application developed using Python Django and Django REST Framework (DRF). The system provides basic banking operations such as customer registration, savings-account creation, account management, deposits, withdrawals, transaction recording, and selected other-bank transactions.

The application uses SQLite for data storage and provides a web interface built with HTML, CSS, and JavaScript. REST APIs are provided for retrieving account information and processing transactions.

2. Technology Stack

Technology	Purpose
Python 3.10.0	Programming language
Django 5.2.17	Web application framework
Django REST Framework	REST API development
SQLite	Database
HTML	Web-page structure
CSS	User-interface styling
JavaScript	Client-side functionality
Windows	Development environment
Postman	API testing
Web Browser	Application/API testing

Django application: **Accounts**

3. System Objectives

The main objectives of the system are:

- To provide a simple digital banking service.
- To maintain customer information.
- To create and manage savings accounts.
- To perform deposits and withdrawals.
- To automatically update account balances.
- To maintain transaction records.
- To provide account information through REST APIs.
- To support selected transactions involving other banks.

- To validate banking operations and handle errors.
- To provide separate access for Admin/Staff and Customers.

4. System Users

The application supports two primary categories of users.

4.1 Admin/Staff

Admin or bank staff can perform the primary banking-management operations, including:

- Customer registration
- Customer information management
- Savings-account creation
- Account information management
- Deposit processing
- Withdrawal processing
- Transaction management
- Other-bank transaction processing
- Viewing account information

4.2 Customer

Customers can access the banking system through login and use the banking operations provided by the application.

Customer functionality includes:

- Login
- Accessing account information
- Performing supported banking transactions (Deposit, Withdrawal)
- Other-bank transaction functionality

No additional profile-management or password-management functionality is documented because it was not identified as part of the confirmed project scope.

5. Main Modules

The system is organized around the following functional modules:

```
Banking Service Management System
|
+-- User Login
|
+-- Customer Management
|
+-- Account Management
|
+-- Transaction Management
```

```
|  
+-- Other-Bank Transactions  
|  
+-- REST API Services
```

6. User Login Module

The login module provides username/password-based access to the application.

Main functions

- Accept username and password.
- Validate login credentials.
- Allow authorized users to access the application.
- Redirect the user to the appropriate application page after successful login.
- Display an error when authentication fails.

Security

The application uses:

- Username/password validation
- Input validation
- Django CSRF protection for applicable forms
- Session-based web application behavior where applicable through Django

7. Customer Management

The Customer Management module stores customer information required for banking operations.

Customer information

The registration process includes information such as:

- Customer ID
- First name
- Last name
- Date of birth
- Gender
- Phone number
- Email
- Address
- City
- State

- Pincode
- Aadhaar
- PAN
- Created date
- Customer status

Customer registration process

```
Enter Customer Details
  |
  v
Validate Required Fields
  |
  v
Check Existing Customer
  |
  v
Validate Information
  |
  v
Save Customer
  |
  v
Customer Successfully Registered
```

The system also performs duplicate checking to prevent the registration of an already existing customer based on the validation implemented in the application.

8. Account Management

The Account Management module handles the creation and retrieval of customer bank accounts.

The confirmed account type for this project is: **Savings Account**

Account information

The Accounts model contains fields including:

Field	Description
acc_id	Account identifier
acc_no	Account number
acc_holder	Account holder name
cust_id	Associated customer ID
acc_type	Account type
balance	Current account balance
created_at	Account creation date

Account creation process

```
graph TD
    A[Select Customer] --> B[Validate Customer]
    B --> C[Generate Account Details]
    C --> D[Select Savings Account]
    D --> E[Set Initial Balance]
    E --> F[Create Account]
```

The system validates the customer before creating the associated account.

9. Transaction Management

The Transaction Management module handles financial operations on accounts.

The supported transaction types are:

- Deposit
- Withdrawal
- Other-bank transaction

9.1 Deposit

A deposit increases the current account balance.

Process

```
graph TD
    A[Enter Account Number] --> B[Find Account]
    B --> C[Read Current Balance]
    C --> D[Enter Deposit Amount]
    D --> E["Current Balance + Amount"]
    E --> F[Update Account]
    F --> G[Create Transaction Record]
```

Example

If: Current Balance = ₹5,000, Deposit = ₹2,000

Then: New Balance = ₹7,000

9.2 Withdrawal

A withdrawal reduces the account balance.

Before processing the withdrawal, the system checks whether the requested amount is available.

Process

```
graph TD
    A[Enter Account Number] --> B[Find Account]
    B --> C[Read Current Balance]
    C --> D[Enter Withdrawal Amount]
    D --> E[Check Available Balance]
    E --> F{Sufficient?}
    F -- Yes --> G[Withdraw]
    F -- No --> H[Show Error]
    G --> I[Update Balance]
    I --> J[Create Transaction Record]
```

Insufficient balance

If the withdrawal amount is greater than the current balance, the transaction is rejected and an appropriate error response is returned.

10. Transaction Database

The Transactions model stores transaction information.

Field	Description
transaction_id	Unique transaction identifier
acc_no	Account number
acc_name	Account holder name

Field	Description
transaction_type	Deposit/Withdrawal
amount	Transaction amount
current_balance	Balance before transaction
balance_after	Balance after transaction
transaction_date	Date of transaction

This provides a record of banking operations performed on an account.

11. Other-Bank Transactions

The system also contains functionality for transactions involving another bank.

The general process is:

```
Select Other Bank
    |
    v
Enter Account Information
    |
    v
Request Account Information
    |
    v
Validate Account
    |
    v
Process Transaction
    |
    v
Record Transaction
```

The application communicates with the required API and includes handling for situations such as network/server errors.

12. REST API

The application uses Django REST Framework to expose banking operations through APIs.

The APIs allow external clients such as Postman or a browser to communicate with the Django application.

API flow

```
Client
  |
  v
HTTP Request
  |
  v
```

```
Django URL
|
v
API View
|
v
Database Operation
|
v
Serializer
|
v
JSON Response
```

13. Get Account API

Purpose

Retrieves account information using the account number.

Method

GET

Endpoint

```
http://127.0.0.1:8000/getaccount/<acc_no>/
```

Example

```
http://127.0.0.1:8000/getaccount/936616/
```

Request

No request body is required.

Example response

```
{
  "acc_id": "A001",
  "acc_no": "936616",
  "acc_holder": "Customer Name",
  "cust_id": "C001",
  "acc_type": "Savings",
  "balance": "5000",
  "created_at": "2026-09-21"
}
```

If the account does not exist:

```
{
  "error": "Account not found"
}
```

14. Get All Accounts API

Purpose

Retrieves all accounts stored in the system.

Method

GET

Endpoint

```
http://127.0.0.1:8000/getallaccount/
```

Example response

```
[
  {
    "acc_id": "A001",
    "acc_no": "936616",
    "acc_holder": "Customer One",
    "cust_id": "C001",
    "acc_type": "Savings",
    "balance": "5000",
    "created_at": "2026-09-21"
  },
  {
    "acc_id": "A002",
    "acc_no": "936617",
    "acc_holder": "Customer Two",
    "cust_id": "C002",
    "acc_type": "Savings",
    "balance": "7500",
    "created_at": "2026-09-21"
  }
]
```

15. Create Transaction API

Purpose

Creates a deposit or withdrawal transaction and updates the account balance.

Method

POST

Endpoint

```
http://127.0.0.1:8000/createtransaction/
```

Headers

```
Content-Type: application/json
```

Deposit request

```
{
  "acc_no": "936616",
  "transaction_type": "deposit",
  "amount": "2000"
}
```

Withdrawal request

```
{
  "acc_no": "936616",
  "transaction_type": "withdraw",
  "amount": "1000"
}
```

Successful response

A successful transaction returns transaction-related information according to the API implementation.

Conceptually:

```
{
  "transaction_id": "1001",
  "acc_no": "936616",
  "transaction_type": "deposit",
  "amount": "2000",
  "current_balance": "5000",
  "balance_after": "7000"
}
```

Insufficient balance response

```
{
  "error": "Insufficient balance"
}
```

16. Serializers

Django REST Framework serializers are used to convert Django model objects into JSON-compatible data.

The project uses an account serializer for account API responses.

A transaction serializer is also used for transaction-related data.

The serializer structure follows the Django REST Framework `ModelSerializer` approach.

Example structure:

```
class TransactionSerializer(serializers.ModelSerializer):
    class Meta:
        model = Transactions
        fields = "__all__"
```

The important configuration is **model = Transactions**; using models instead would cause a serializer configuration error.

17. Database Design

The application uses SQLite as its database.

The primary banking entities identified in the project are:

```
Customer
  |
  | cust_id
  v
Accounts
  |
  | acc_no
  v
Transactions
```

Customer

Stores customer personal and contact information.

Accounts

Stores bank-account information associated with customers.

Transactions

Stores financial transactions associated with accounts.

18. Data Relationship

The logical relationship can be represented as:

```
CUSTOMER
  |
  | Customer ID
  |
  v
ACCOUNT
  |
  | Account Number
  |
  v
TRANSACTION
```

A customer can have an associated bank account, and transactions are associated with an account through the account number.

19. Validation and Error Handling

Validation is important because banking operations involve financial data.

The application includes validation for situations such as:

Customer registration

- Required fields
- Duplicate customer information
- Invalid/missing form information

Account creation

- Customer existence
- Required account information

Transactions

- Account existence
- Transaction type
- Transaction amount
- Insufficient balance

API operations

- Account not found
- Invalid requests
- Server/network errors for external-bank communication

20. Error Handling Examples

Account not found

```
{
  "error": "Account not found"
}
```

Insufficient balance

```
{
  "error": "Insufficient balance"
}
```

Invalid transaction type

The API validates the transaction type and rejects unsupported transaction operations.

External API/server error

The other-bank transaction functionality includes handling for communication/server failures.

21. API Testing

The APIs are tested using:

- Postman
- Web Browser

Postman testing workflow

```
Open Postman
  |
  v
Select HTTP Method
  |
  v
Enter API URL
  |
  v
Enter Request Body
  |
  v
Send Request
  |
  v
Check Status/JSON Response
```

For example:

```
POST
http://127.0.0.1:8000/createtransaction/

with:

{
  "acc_no": "936616",
  "transaction_type": "deposit",
  "amount": "2000"
}
```

22. Running the Project

The project is currently designed to run on localhost.

Step 1 – Install Python

Python 3.10.0 is used for the project. Verify installation:

```
python --version
```

Expected:

```
Python 3.10.0
```

Step 2 – Install Django

```
pip install django
```

Step 3 – Install Django REST Framework

```
pip install djangorestframework
```

Step 4 – Navigate to the project

```
cd <project-directory>
```

Step 5 – Apply migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 6 – Start the development server

```
python manage.py runserver
```

The application can then be accessed through:

```
http://127.0.0.1:8000/
```

23. Project Structure

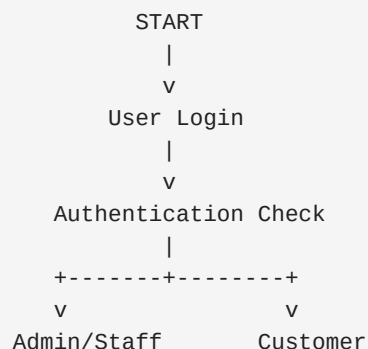
A typical structure for the project can be represented as:

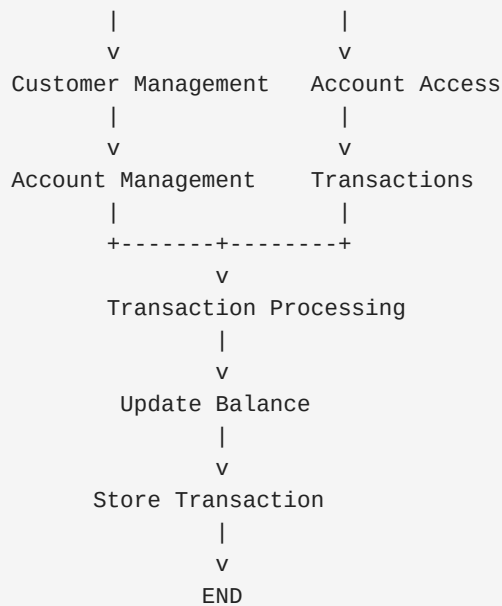
```
Banking_Service/
|
+-- manage.py
|
+-- Banking_Service/
|   +-- settings.py
|   +-- urls.py
|   +-- ...
|
+-- Accounts/
|   +-- models.py
|   +-- views.py
|   +-- serializers.py
|   +-- urls.py
|   +-- admin.py
|   +-- ...
```

The Accounts application contains the main banking functionality.

24. Application Workflow

The overall workflow of the system is:





25. Security

The application includes basic security measures appropriate to the implemented application.

Authentication

Users access the system through username/password validation.

CSRF protection

Django's CSRF protection is used for applicable web forms to help protect against unauthorized cross-site form submissions.

Input validation

User input is validated before performing important operations such as registration, account creation, and transactions.

Banking validation

Withdrawal operations verify the available balance before modifying the account.

Note: Advanced mechanisms such as JWT authentication, OAuth, or token-based API authentication are not included in this documentation because they were not confirmed as implemented.

26. Advantages

The system provides several practical benefits:

- Centralized customer information management
- Digital account management

- Automated balance calculation
- Transaction record maintenance
- REST API support
- Deposit and withdrawal validation
- Insufficient-balance checking
- Other-bank transaction support
- Simple web-based interface
- Easy local development and testing

27. Limitations

The current implementation has some limitations:

- The application currently runs only on localhost.
- SQLite is suitable for development and smaller applications but is not generally preferred for a production-scale banking system.
- The project provides basic authentication rather than a complete production banking security architecture.
- Advanced authorization and role-based permissions may require further development.
- The project does not represent a complete real-world banking infrastructure.

28. Future Enhancements

Possible future improvements include:

- **Production database:** Migrate from SQLite to PostgreSQL or MySQL for larger-scale deployment.
- **Advanced authentication:** Implement stronger authentication and authorization mechanisms.
- **Role-based access control:** Provide different permissions for administrators, staff, and customers.
- **Transaction history:** Provide a dedicated customer transaction-history page.
- **Account statements:** Generate downloadable PDF account statements.
- **Email/SMS notifications:** Notify customers about successful deposits and withdrawals.
- **Online deployment:** Deploy the application to a cloud/server environment.
- **Enhanced API security:** Add authenticated and permission-controlled APIs.
- **Improved validation:** Add stronger validation for financial amounts and customer information.
- **Dashboard:** Provide dashboards showing account balances and transaction summaries.

29. Testing Checklist

Test Case	Expected Result
Valid login	User accesses system

Test Case	Expected Result
Invalid login	Error displayed
Valid customer registration	Customer created
Duplicate customer	Registration rejected
Valid account creation	Savings account created
Existing customer validation	Account creation allowed
Get valid account	Account JSON returned
Get invalid account	Account-not-found error
Get all accounts	Account list returned
Deposit	Balance increases
Withdrawal with sufficient balance	Balance decreases
Withdrawal with insufficient balance	Transaction rejected
Invalid transaction type	Error returned
Other-bank API available	Transaction processed
Other-bank API unavailable	Error handled

30. API Summary

Operation	Method	Endpoint
Get Account	GET	/getaccount/
Get All Accounts	GET	/getallaccount/
Create Transaction	POST	/createtransaction/

Localhost base URL

http://127.0.0.1:8000/

Therefore:

```
GET http://127.0.0.1:8000/getaccount/936616/
GET http://127.0.0.1:8000/getallaccount/
POST http://127.0.0.1:8000/createtransaction/
```

31. Conclusion

The Banking Service Management System is a Django-based web application that implements fundamental banking-service operations. It manages customers and savings accounts, supports deposits and withdrawals, maintains transaction information, and exposes REST APIs for account and transaction

operations.

The combination of Django, Django REST Framework, SQLite, HTML, CSS, and JavaScript provides a practical foundation for developing and testing the application locally. The system can be further extended with stronger security, production database support, advanced user permissions, notifications, account statements, and cloud deployment.

Quick Project Summary

Attribute	Detail
Project	Banking Service Management System
Framework	Django 5.2.17
Language	Python 3.10.0
API	Django REST Framework
Database	SQLite
Frontend	HTML, CSS, JavaScript
Django App	Accounts
Users	Admin/Staff and Customers
Account	Savings
Transactions	Deposit, Withdrawal, Other-bank transaction
Testing	Postman + Browser
Deployment	Localhost
Development OS	Windows